LAPORAN PRAKTIKUM MIKROKONTROLER

KONVERSI DATA

BAHASA ASSEMBLY



Elita Lestari Setianto NIM 201344008

Kelas 3 NK

Tanggal Percobaan : Rabu, 31 Agustus 2022

Tanggal Penyerahan Laporan : Selasa, 06 September 2022

Instruktur : 1. Ferry Satria, BSEE, M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2022

A. TUJUAN

1. Untuk memenuhi tugas mata kuliah Mikrokontroler.
2. Mahasiswa dapat mengetahui bahasa pemograman Assembly.
3. Mahasiswa mampu melakukan instalasi dan menggunakan *software* AVR Studio 4 untuk simulasi pemrogaman mikrokontroler.
4. Mahasiswa mampu memahami konsep Bahasa Pemrograman Assembly dan menjalankannya di *software* AVR Studio 4.
5. Mahasiswa mampu memahami konsep dan membuat program input data serta konversi data dari heksadesimal ke desimal dan sebaliknya pada *software* AVR Studio 4.

B. PENDAHULUAN

1. Mikrokontroler

Mikrokontroler merupakan suatu terobasan teknologi mikroprosesor dan mikrokomputer yang merupakan teknologi semikonduktor dengan kandungan transistor yang lebih banyak namun hanya membutuhkan ruang yang sangast kecil, Lebih lanjut, mikrokontroler merupakan system computer yang mempunyai satu atau beberapa tugas yang sangat spesifik, berbeda dengan PC (*Personal Computer*) yang memiliki beragam fungsi.

Tidak seperti sistem komputer yang mampu menangani berbagai macam program aplikasi, mikrokontrler hanya bisa digunakan untuk suatu aplikasi tertentu saja, perbedaan lainnya terletak pada perbandingan RAM dan ROM. Pada sistem komputer perbandingan RAM dan ROM nya besar, artinya program-program penggunba disimpan dalam ruang RAM yang relative besar, sedangkan rutin-rutin antar muka perangkat keras disimpan dalam ruang ROM yang kecil, Sedangkan pada mikrokontroler, perbandingan ROM dan RAM â€“nya yang besar, artinya program kontrol disimpan dalm ROM (bias *Masked* ROM atau *Flash* PEROM) yang ukurannya relatif lebih besar, sedangkan RAM digunakan sebagai tempat penyimpanan sementara , termasuk register-register yang digunakn pada mikrokontroler yang bersangkutan.

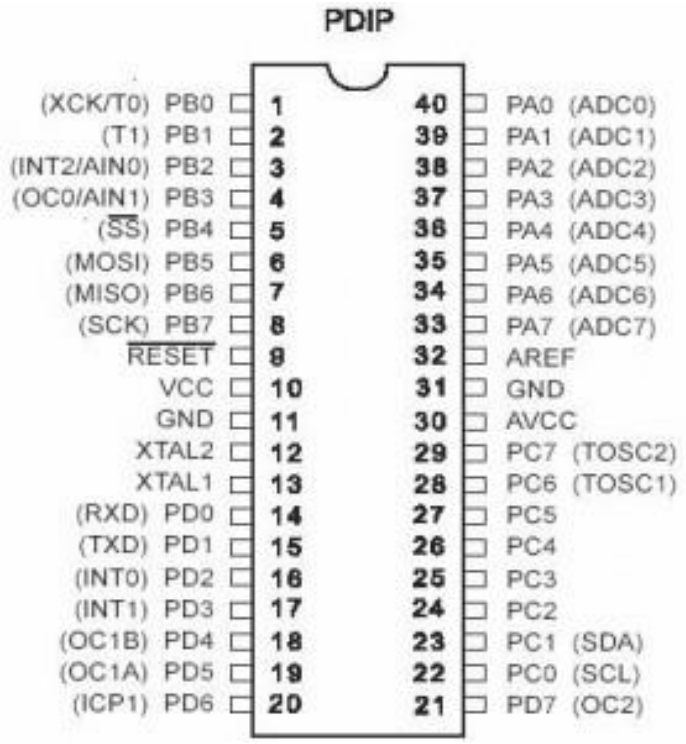
2. Mikrokontroler ATMega8535

Mikrokontroler merupakan keseluruhan sistem komputer yang dikemas menjadi sebuah *chip* di mana di dalamnya sudah terdapat Mikroprosesor, I/O, Memori bahkan ADC, berbeda dengan Mikroprosesor yang berfungsi sebagai pemroses data (Heryanto, dkk, 2008:1).

Mikrokontroller AVR (*Alf and Vegardâ€™s Risc processor*) memiliki arsitektur 8 bit, dimana semua instruksi dikemas dalam kode 16-*bit* dan sebagian besar instruksi dieksekusi dalam 1 siklus *clock* atau dikenal dengan teknologi RISC (*Reduced Instruction Set Computing*). Secara umum, AVR dapat dikelompokan ke dalam 4 kelas, yaitu keluarga AT90Sxx, keluarga ATMega dan AT86RFxx. Pada dasarnya yang membedakan masing-masing adalah kapasitas memori, *peripheral* dan fungsinya (Heryanto, dkk, 2008:1). Dari segi arsitektur dan instruksi yang digunakan, mereka bisa dikatakan hampir sama. Berikut ini gambar Mikrokontroler Atmega8535.



Gambar 2.1 *Mikrokontroler ATMega8535*



Gambar 2.2 Konfigurasi Pin ATMega8535

Â **Konfigurasi Pin ATMega8535**

Secara umum konfigurasi dan fungsi pin ATMega8535 dapat dijelaskan sebagai berikut

1 **VCC** Input sumber tegangan (+)

2 **GND** Ground (-)

3 **Port A (PA7 â€¦ PA0)** Berfungsi sebagai input analog dari ADC (Analog to Digital Converter). Port ini juga berfungsi sebagai port I/O dua arah, jika ADC tidak digunakan.

4 **Port B (PB7 â€¦ PB0)** Berfungsi sebagai port I/O dua arah. Port PB5, PB6 dan PB7 juga berfungsi sebagai MOSI, MISO dan SCK yang dipergunakan pada proses downloading. Fungsi lain port ini selengkapnya bisa dibaca pada buku petunjuk â€• AVR ATMega8535â€• .

5 **Port C (PC7 â€¦ PC0)** Berfungsi sebagai port I/O dua arah. Fungsi lain port ini selengk apnya bisa dibaca pada buku petunjuk â€• AVR ATMega8535â€• .

6 **Port D (PD7 â€¦ PD0)** Berfungsi sebagai port I/O dua arah. Port PD0 dan PD1 juga berfungsi sebagai RXD dan TXD, yang dipergunakan untuk komunikasi serial. Fungsi lain port ini selengkapnya bisa dibaca pad a buku petunjuk â€• AVR ATMega8535â€• .

7 **RESET** Input reset.

8 **XTAL1** Input ke amplifier inverting osilator dan input ke sirkuit clock internal.

9 Â Â **XTAL2** Output dari amplifier inverting osilator.

10 **AVCC** Input tegangan untuk Port A dan ADC.

11 **AREF** Tegangan referensi untuk ADC.

Fitur Mikrokontroler ATMega8535

Adapun kapabilitas detail dari ATmega8535 adalah sebagai berikut,

1 Sistem mikroprosesor 8 bit berbasis *RISC* dengan kecepatan maksimal 16 MHz.

2 Kapabilitas memori *flash* 8 KB, *SRAM* sebesar 512 byte, dan *EEPROM* (*Electrically Erasable Programmable Read Only Memori*) sebesar 512 byte.

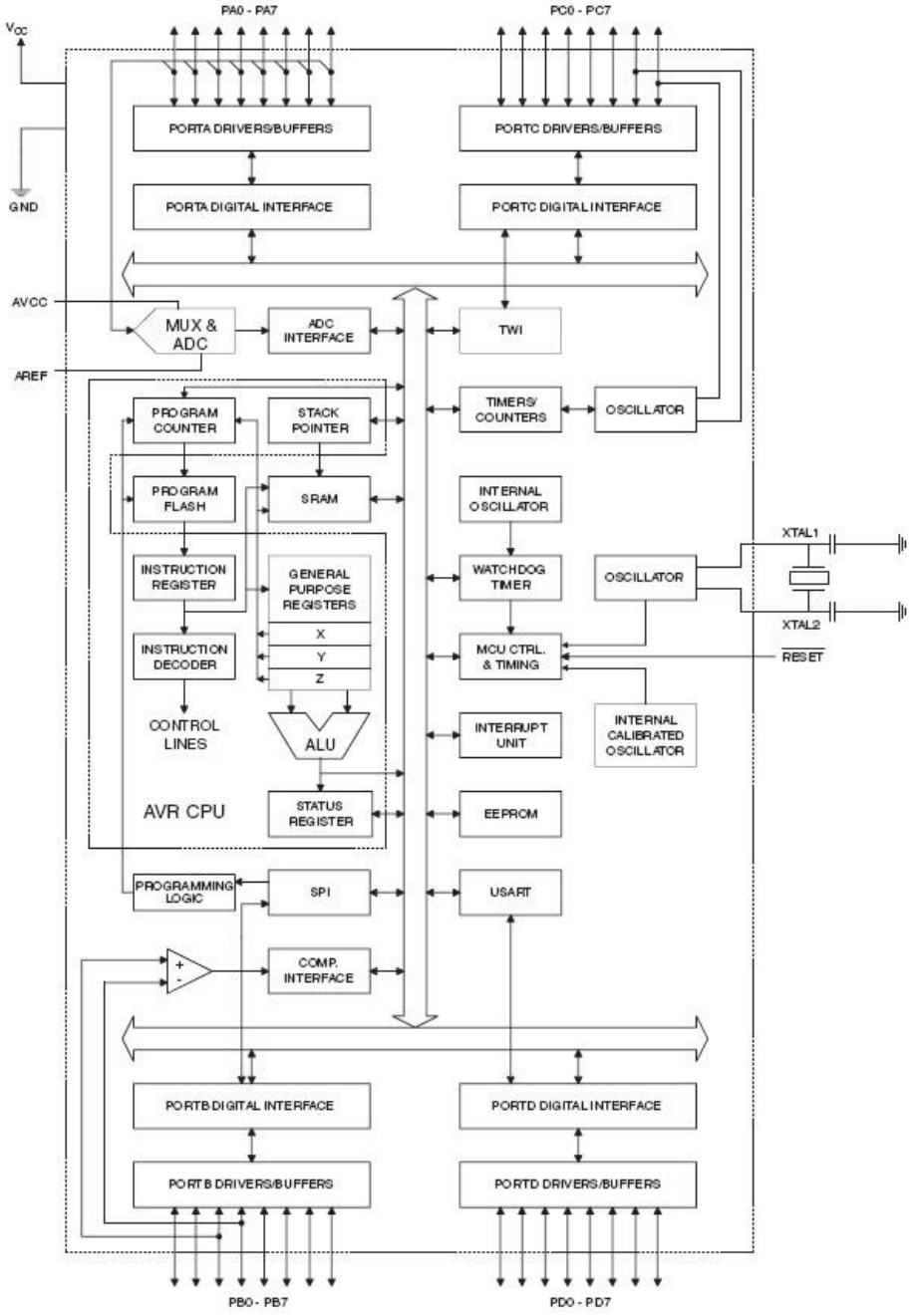
3 *ADC* internal dengan fidelitas 10 bit sebanyak 8 *channel*.

4 Portal komunikasi serial (*USART*) dengan kecepatan maksimal 2,5 Mbps.

5 Enam pilihan mode *sleep* untuk menghemat penggunaan daya listrik.

Arsitektur ATMega8535

Blok-Diagram



Gambar 2.3 *Blok diagram fungsional ATmega8535*

Dari gambar blok diagram tersebut dapat dilihat bahwa ATMega8535 memiliki bagian-bagian sebagai berikut :

1. Saluran I/O sebanyak 32 buah, yaitu *Port* A,*Port* B,*Port* C dan *Port* D.

2. ADC 8 *channel* 10 bit.

3. Tiga buah *Timer*/*Counter* dengan kemampuan pembanding.

4. CPU yang terdiri atas 32 buah *register*.

5. *Watchdog timer* dengan osilator *internal*.

6. SRAM sebesar 512 byte.

7. Memori *Flash* sebesar 8 KB dengan kemampuan *Read While Write*.

8. *Interrupt internal* dan *eksternal*

9. *Port* antarmuka SPI (*Serial Peripheral Interface*).

10. EEPROM sebesar 512 byte yang dapat diprogram saat operasi.

11. Antarmuka komparator analog.

12. *Port* USART untuk komunikasi serial

3. Software AVR Studio

Software CodeVisionAVR merupakan perangkat lunak kompailer C dan merupakan pembangkit program otomatis. Software CodeVisionAVR didesain untuk mikrokontroller keluarga atmel AVR. Software CodeVisionAVR dapat berjalan pada sistem operasi windows 95, 98, Me, NT 4, 2000 dan windows XP. Kompailer C melakukan pendekatan elemen dari bahasa ANSI C sesuai dengan arsitektur dari AVR, dengan beberapa fitur yang ditambahkan. Integrated Development Environment (IDE)

mendukung software AVR Chip In-System Programmer sehingga memungkinkan untuk melakukan transfer secara otomatis dari program ke chip mikrokontroller setelah berhasil dikompilasi atau diassembly. Software InSystem Programmer didesain untuk dapat bekerja dan mendukung tipe chip AVR STK500/AVRISP/AVRProg, Kanda Systems STK200+/300, Dontronics DT006, Vogel Elektronik VTEC-ISP, Futurlec JRAVR dan MicroTronics ATCPU/Mega2000 programmers/development boards. Software CodeVisionAVR juga mempunyai fasilitas CodeWizardAVR, yang digunakan untuk membantu menuliskan dengan cepat secara otomatis kedalam program. Dengan CodeWizardAVR membantu kita untuk membuat kerangka program dan menuliskan fungsi-fungsi:

A. Mengatur External memory
B. Mengidentifikasi sumber reset Chip
C. Inisialisai Port Input/Output8]
D. Inisialisasi External Interrupts
E. Inisialisasi Timers/Counters
F. Inisialisasi Watchdog Timer
G. Inisialisasi UART (USART) and interrupt driven buffered serial communication
H. Inisialisasi Analog Comparator
I. Inisialisasi ADC
J. Inisialisasi antarmuka SPI
K. Inisialisasi antarmuka Two Wire
L. Inisialisasi antarmuka CAN
M. Inisialisasi I2C Bus, LM75 Temperature Sensor, DS1621 Thermometer/Thermostat dan PCF8563, PCF8583, DS1302, DS1307 Real Time Clocks
N. Inisialisasi 1 Wire Bus and DS1820, DS18S20 Temperature Sensors
O. Inisialisasi LCD module

4. Konversi Data

Konversi data merupakan pengubah format data komputer dari satu format ke format lainnya yang akan di data lalu dikodekan atau dioperasikan dengan berbagai cara serta dibuat berdasarkan standar tertentu.[1] Ada berbagai cara untuk mengonversi suatu dokumen

1. Desimal ke Biner

Cara mengkonversi bilangan desimal ke biner adalah dengan cara membagi bilangan desimal dengan 2 (basis bilangan biner) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan biner. Nilai konversinya adalah urutan sisa hasil bagi dari yang paling akhir.

Contoh :

$88_{(10)} = \ldots_{(2)}$

- 88 : 2 = 44 sisa 0
- 44 : 2 = 22 sisa 0
- 22 : 2 = 11 sisa 0
- 11 : 2 = 5 sisa 1
- 5 : 2 = 2 sisa 1
- 2 : 2 = 1 sisa 0
- 1 : 2 = 0 sisa 1

Maka hasil Konversinya = 1011000 (ditulis dengan urutan dari bawah ke atas)

2. Desimal ke Octal

Cara mengkonversi bilangan desimal ke Oktal adalah dengan cara membagi bilangan desimal dengan 8 (basis bilangan oktal) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan oktal. Nilai konversinya adalah urutan sisa hasil bagi dari yang paling akhir.

Contoh :

1402(10) = … (8)

- 1402/8 = 175 sisa 2
- 175/8 = 21 sisa 7
- 21/8 = 2 sisa 5

· 2/8 = 0 sisa 2

Maka hasil konversinya = 2572 (ditulis dari bawah)

3. Desimal ke Hexadecimal

Cara mengkonversi bilangan desimal ke hexadesimal adalah dengan cara membagi bilangan desimal dengan 16 (basis bilangan hexadesimal) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan heksadesimal. Apabila sisa bagi > 9 maka angkanya dirubah menjadi huruf. Untuk sisa bagi berjumlah 10 = A, 11 = B, 12 = C, 13 = D, 14 = E, 15 = F.

Contoh :

19889(10) = … (16)

· 19889/16 = 1243 sisa 1

· 1243/16 = 77 sisa 11 (B)

· 77/16 = 4 sisa 13 (D)

· 4/16 = 0 sisa 4

Maka hasil konversinya = 4DB1 (ditulis dari bawah)

TABEL KONVERSI BILANGAN

Decimal (x_{10})	Biner (x_2)	Octal (x_8)	Hexa (x_{16})
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

4. Biner ke Octal

Cara mengkonversi bilangan biner ke oktal yakni dengan mengelompokan bilangan biner menjadi 3 kelompok **dimulai dari bilangan biner yang paling kanan**. Setelah dikelompokan barulah kita dapat mengkonversi menjadi bilangan Oktal.
Contoh :

11001101(2) = … (8) ===> 011 001 101

011 = 3 (lihat tabel konversi di atas)

001 = 1

101 = 5

Hasil Konversi : 315 (ditulis dari atas ke bawah)

5. Biner ke Desimal

Cara mengkonversi bilangan biner ke desimal adalah dengan cara mengalikan satu persatu bilangan dengan 2 (basis bilangan biner) pangkat 0, pangkat 1 dan seterusnya sesuai dengan banyaknya bilangan biner yang akan di konversi dan perhitungannya dimulai dari bilangan biner yang paling kanan.

Contoh :

$00011_{(2)} = \ldots_{(10)}$

$= (1x2^0) + (1x2^1) + (0x2^2) + (0x2^3) + (0x2^4)$

$= 1 + 2 + 0 + 0 + 0$

$= 3$

Hasil Konversi : 3

6. Biner ke Hexadecimal

Cara mengkonversi bilangan biner ke hexadesimal tekniknya hampir sama dengan cara konversi bilangan biner ke oktal. **Yang membedakan ada pada pengelompokan bilangan binernya, pada bilangan oktal dalam satu kelompok terdiri dari 3 bit bilangan biner, sedangkan pada hexadesimal dalam satu kelompok terdiri dari 4 bit bilangan biner**.
Contoh :

$10100_{(2)}$ = ... (16)

· 0001 = 1 (lihat tabel konversi di atas)

· 0100 = 4

Hasil Konversi = 14 (ditulis dari atas ke bawah)

7. Octal Ke Desimal

Cara mengkonversi bilangan oktal ke desimal adalah dengan cara mengalikan satu persatu bilangan dengan 8 (basis bilangan oktal) dengan pangkat 0, 1 dan seterusnya dimulai dari bilangan oktal yang paling kanan. Kemudian hasil dari semua pengalian dijumlahkan.

Contoh :

$62_{(8)}$ = ... (10)

· $2x8^0 = 2$

· $6x8^1 = 48$

2+48 = 50

Hasil Konversi = 50

8. Octal Ke Biner

Cara mengkonversi bilangan oktal ke biner adalah dengan **memecah** terlebih dahulu bilangan oktal kedalam satuan bilangan, kemudian masing-masing bilangan diubah kedalam bentuk 3 bit bilangan biner, dengan cara membagi setiap satuan bilangan tersebut dengan 2 (basis bilangan biner). Jika hasil konversi hanya menghasilkan 2 digit bilangan biner, maka harus ditambahkan 0 di sebelah kirinya, supaya bilangan binernya menjadi 3 digit (bit).
Contoh :

$145_{(8)}$ = ... (2)

1 = 001 (lihat tabel konversi)

4 = 100

5 = 101

Hasil Konversi = 001100101 (ditulis dari atas ke bawah)

9. Octal Ke Hexadecimal

Cara mengkonversi bilangan oktal ke hexadesimal terdiri dari dua tahap yaitu:

- Pertama, mengkonversi terlebih dahulu setiap bit bilangan oktal ke bilangan biner

- Kedua, hasil konversi ke bilangan biner kemudian di konversikan ke bilangan hexadesimal

Singkatnya seperti ini: Oktal --> Biner --> Hexadesimal.

Contoh :

$46_{(8)}$ = ... (16)
Diubah ke Biner

4 = 100

6 = 110

Diubah Ke Hexadecimal

0100 = 2

0110 = 6

Hasil Konversi = 26

10. Hexadecimal ke Biner

Sama dengan cara konversi bilanga octal ke biner, **bedanya kalau bilangan octal binernya harus 3 bit, sedangkan bilangan hexadesimal binernya terdiri dari 4 bit**. Misal kita konversi 2 hexa menjadi biner hasilnya bukan 10 melainkan 0010
Contoh :

$9A_{(16)}$ = ... (2)
9 = 1001 (lihat tabel konversi)

A/10 = 1010

Hasil Konversi = 10011010 (ditulis dari atas)

11. Hexadecimal ke Desimal

Cara mengkonversi bilangan biner ke desimal adalah dengan mengalikan satu-satu bilangan dengan 16 (basis hexa) pangkat 0 atau 1 atau 2 dst., dimulai dari bilangan paling kanan. Kemudian hasilnya dijumlahkan.

Contoh :

$7A_{(16)} = ..._{(10)}$

$= (7x16^1) + (10x16^0)$

$= 112 + 10$

$= 122$

Hasil Konversi = 122

12. Hexadecimal ke Oktal

Begitu juga dengan konversi hexa desimal ke octal yakni dengan mengubah bilangan hexa ke bilangan biner, kemudian diubah menjadi bilangan octal. Ringkasnya: hexa->biner->octal

Contoh :

$AF_{(16)} = ..._{(10)}$

Diubah ke Biner

A/10 = 1010 (lihat tabel konversi)

F/15 = 1111 ==> hasil konversinya 10101111

Diubah ke Oktal ==> 010 101 111

010 = 2 (lihat tabel konversi)

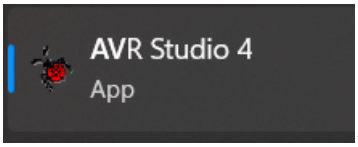
101 = 5

111 = 7

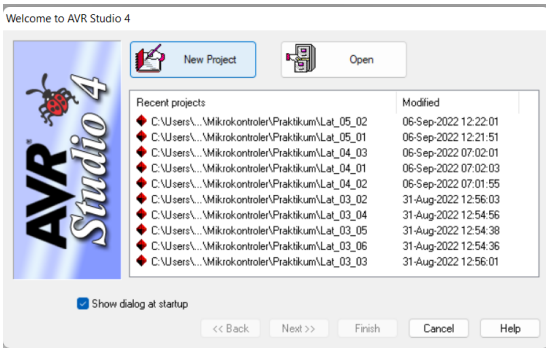
Hasil Konversi = 257 (ditulis dari atas)

C. LANGKAH KERJA

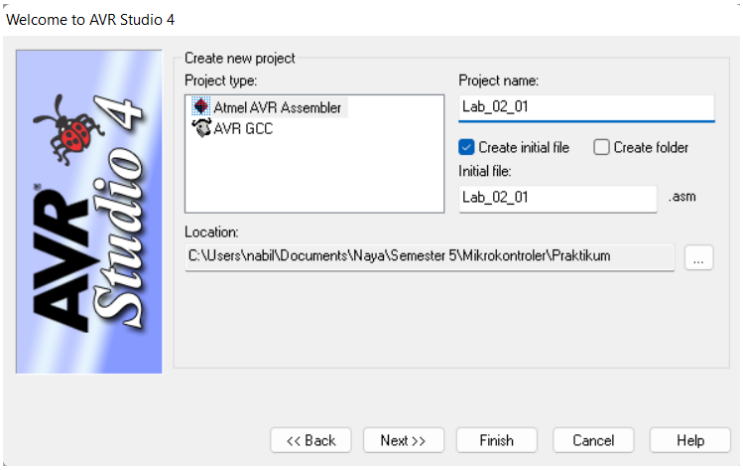
1. *Install* AVR Studio 4
2. Membuka aplikasi

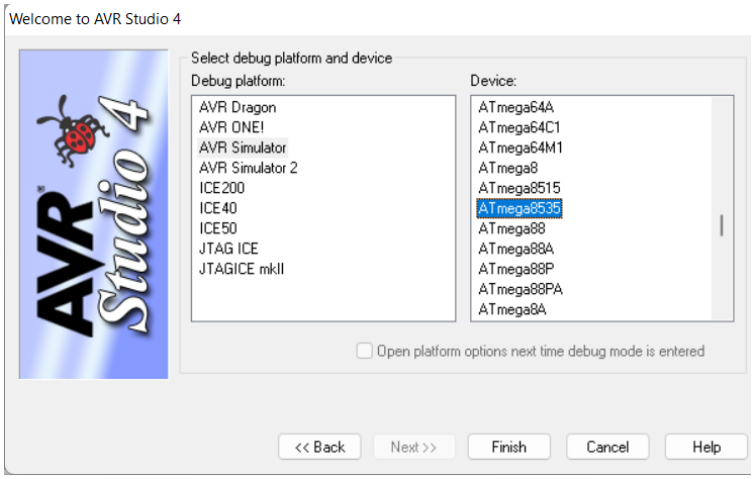


3. Membuat *project* baru

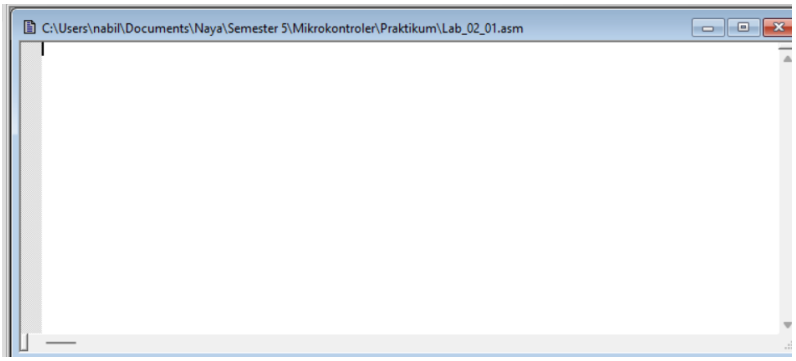


4. Memberikan nama pada project tersebut → klik **next** → klik **AVR simulator** → klik **ATmega8535** → klik **Finish**





5. Menuliskan program pada *sheet* yang disediakan.



6. Kemudian, lakukan *running* program

 , F11

D. PEMBAHASAN DAN ANALISIS

1. Pembahasan

   A. Konversikan bilangan Heksadesimal (2 digit) berada pada register R20 menjadi desimal. Tempatkan hasil konversi pada Reg.R22:R21

```
 .include "m8535def.inc"
.org 0x00
rjmp main
    ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16
ldi r17,high(ramend)
out sph,r17

ldi r20,0XF0 ; bebas angkanya
mov r25,r20
andi R25,0XF0 ; AMBIL JADI 16 AN, PAKE LSR
lsr R25
lsr R25
lsr R25
lsr R25
mov r19,r25
add r19,r30;karena flagnya udh terpengaruh sama r19 (carry)
rcall daa
mov r21,r19
ldi r16,4
lup:mov r19,r21
    add r19,r19
    rcall daa
    mov r21,r19
    mov r19,r22
    adc r19,r19
    rcall daa
```

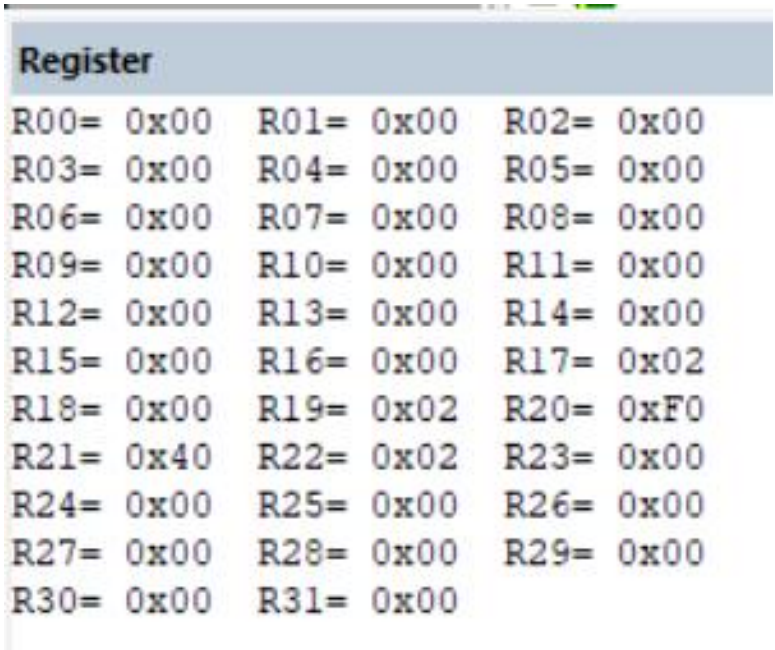
```
        mov r22,r19
        dec r16
        BRNE lup
    mov r25,r20
    andi r25,0X0f
    mov r19,r25
    add r19,r30
    rcall daa
    mov r25,r19
    mov r19,r21
    add r19,r25
    rcall daa
    mov r21,r19
    mov r19,r22
    adc r19,r30
    rcall daa
    mov r22,r19


    akhir: rjmp akhir

    ; subrutin unt koreksi hasil penjumlahan mjd. BCD.2 dgt.
    ; yang dilaksanakan mll. r19 sbg. acc.:
    daa:    push r16;amankan isi reg. ke stack
            push r17
            push r21
            push r24
            in r16,sreg; amankan sreg
            in r17,sreg
            ldi r24,0 ; r24 hanya sbg status dr carry flag
            adc r24,r24; status carry flag sebelum koreksi
    ; ambil, amankan kembali sreg, uji half carry
            out sreg,r17
```

```
        brhc lup4; lompat ke lup4 jika H=0
        rcall fk_ln; satuan + 06 jika H=1
    ;   rjmp lup5
lup4:   mov r21,r19
        andi r21,$0f; ambil dgt.satuan
        cpi r21,$0a ; > 9
        brcs lup5; lompat ke lup5 jika < 9
        rcall fk_ln; satuan + 06 jika > 9
lup5:   out sreg,r17; ambil, amankan kembali sreg, uji carry
        brcc lup6; lompat ke lup6 jika C=0
        rcall fk_hn; puluhan + 60 jika C=1
    ;   rjmp lup7
lup6:   mov r21,r19
        andi r21,0xf0; ambil dgt.puluhan
        cpi r21,0xa0
        brcs lup7; lompat ke lup7 jika pul < a
        rcall fk_hn; pul + 60 jika > 9
lup7:   out sreg,r16; mengatur carry flag sblm keluar:
        clc; C = 0
        in r16,sreg ;amankan sreg stlh c=0
        cpi r24,0 ;menguji status
        breq lup8
        out sreg,r16;ambil sreg, set c
        sec; C = 1
        in r16,sreg; amankan sreg stlh c=1
lup8:
        out sreg,r16;status akhir
        pop r24;kembalikan isi reg. dr stack
        pop r21
        pop r17
        pop r16
        ret
fk_ln:  ldi r21,06; koreksi dgt.satuan
```

```
        out sreg,r16
        add r19,r21
        in r16,sreg
        brcc tuh
        ldi r21,0x01
        add r17,r21
tuh:    out sreg,r16
        adc r24,r24
        ret
fk_hn:      ldi r21,$60; koreksi dgt.puluhan
        out sreg,r16
        add r19,r21
        in r16,sreg
        adc r24,r24; amankan carry pd r24
        ret ; akhir dari prog. subrutin.
```



Register

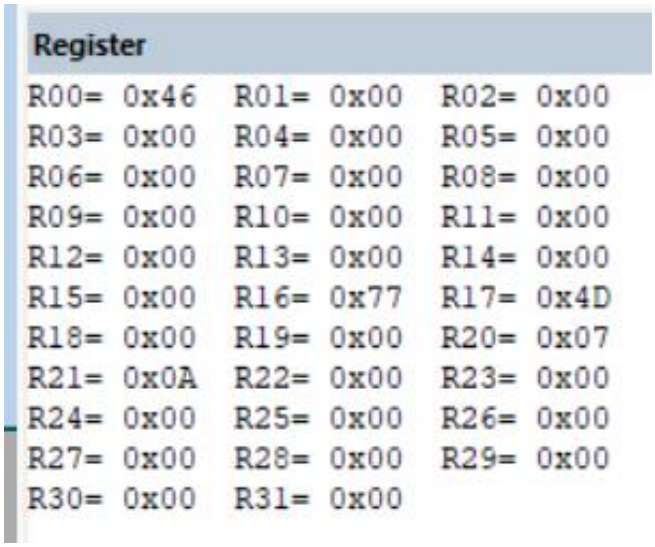
R00= 0x00	R01= 0x00	R02= 0x00
R03= 0x00	R04= 0x00	R05= 0x00
R06= 0x00	R07= 0x00	R08= 0x00
R09= 0x00	R10= 0x00	R11= 0x00
R12= 0x00	R13= 0x00	R14= 0x00
R15= 0x00	R16= 0x00	R17= 0x02
R18= 0x00	R19= 0x02	R20= 0xF0
R21= 0x40	R22= 0x02	R23= 0x00
R24= 0x00	R25= 0x00	R26= 0x00
R27= 0x00	R28= 0x00	R29= 0x00
R30= 0x00	R31= 0x00	

B. Konversikan bilangan desimal (2 digit) berada pada register R16 menjadi hexadesimal. Tempatkan hasil konversi pada Reg.R17

```
 .include "m8535def.inc"
.org 0x00
rjmp main
    ; PAKAI PROJECT "ROOT"
main :
ldi r16,low(ramend)
out spl,r16
ldi r17,high(ramend)
out sph,r17

ldi r16,0X77
mov r20,r16
andi R20,0XF0
lsr R20
lsr R20
lsr R20
lsr R20
ldi r21,0x0A
mul r20,r21
mov r17,r0
mov r20,r16
andi r20,0x0f
add r17,r20


akhir: rjmp akhir
```



C. Konversikan bilangan desimal (4 digit) berada pada data area menjadi hexadesimal. Tempatkan hasil konversi pada Reg.R17

```
 .include "m8535def.inc"
.org 0x00
rjmp main
    ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20

ldi r28,0x60          ; input data dari data area
ld r17,y+
ld r16,y

mov r18,r17
mov r19,r16
andi r18,0xf0         ; mengambil nilai ribuan (d1)
lsr r18
lsr r18
```

```
lsr r18
lsr r18
ldi r31,0x0a        ; pengali
mul r18,r31             ; d1*0a
mov r21,r0
mov r22,r1
mov r18,r17
andi r18,0x0f       ; mengambil nilai ratusan (d2)
add r21,r18             ; d1*0a + d2
adc r22,r30

mul r21,r31             ; (((d1*0a)+d2)*0a)
mov r21,r0
mov r22,r1              ; (((d1*0a)+d2)*0a)
andi r19,0xf0       ; mengambil nilai d3
lsr r19
lsr r19
lsr r19
lsr r19
add r21,r19             ; (((d1*0a)+d2)*0a)+d3
adc r22,r30

mul r21,r31
mov r2,r0
mov r3,r1

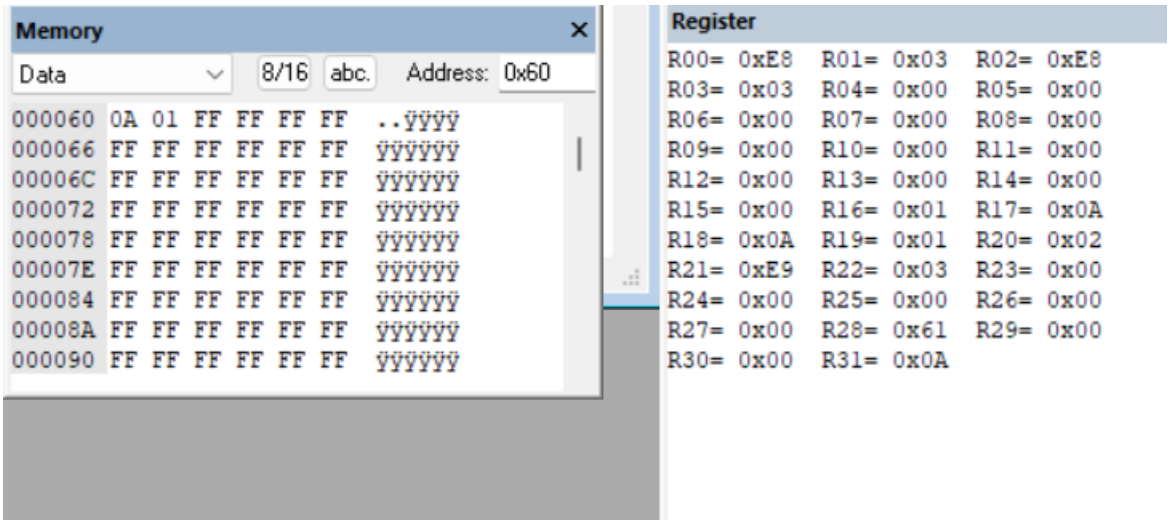
mul r22,r31
rcall gnd
add r0,r2
adc r1,r3               ; ((((d1*0a)+d2)*0a)+d3)*0a
mov r21,r0
mov r22,r1              ; ((((d1*0a)+d2)*0a)+d3)*0a
```

```
        mov r19,r16
        andi r19,0x0f           ; mengambil nilai d4

        add r21,r19
        adc r22,r30                     ; (((((d1*0a)+d2)*0a)+d3)*0a)+d4

        akhir: rjmp akhir

        gnd:                            ; subrutin untuk menggeser sebanyak 8 kali ke kiri
        ldi r25,8
        l1:  add r0,r0          ; register r0 dan r1
             adc r1,r1
             dec r25
             brne l1
```



D. Analisis

Pada praktikum kali ini, terdapat 3 program yang perlu dilakukan, namun sebelumnya ada beberapa perintah yang biasanya selalu ada agar dapar membuat suatu program berjalan. Praktikum kali ini menggunakan *software* simulasi mikrokontroler yaitu ATmega 8535 dan menggunakan konsep bahasa Assembly yang dimana bahasa pemrograman ini termasuk ke bahasa tingkat renda (*low level language*) atau bisa dibilang bahasa mesin. Simulator ATmega 8535 bukan simulator yang *sensitive case*

sehingga besar kecilnya huruf ketika penulisan program tidak akan berpengaruh kepada berjalannya suatu program (kecuali suatu nama subrutin, harus sama antara nama subrutin dengan subrutin yang dipanggil).

Dari setiap program yang digunakan, terdapat beberapa baris perintah "wajib" agar program dapat berjalan. Penjelasan dari tiap baris program yang digunakan tersebut sebagai berikut:

Perintah/Sintaks	**Deskripsi**
.include"m8535def.inc"	Mendefinisikan mikrokontroler yang direferensikan untuk program yaitu ATmega8535.
.org 0x0000	Mendefinisikan alamat awal program.
rjmp main	Melakukan *jump* (*relative jump*) ke label main *procedure* (prosedur utama).
ldi r16,low(ramend)	Menyiapkan *stack* (*stack pointer low* dan *stack pointer high*) untuk menerima data-data yang akan diletakkan/digunakan. Menempatkan 8bit pertama dan beberapa bit kedua ke R16, "low" berarti 8bit pertama di sebelah kanan dan "high" berarti beberapa bit kedua. "ramend" berarti alamat ram terakhir.
out spl,r16	
ldi r16,high(ramend)	
out sph,r16	
Akhir : rjmp Akhir	Untuk mengakhiri proses *running* programnya.

Pada program pertama yaitu mengonversi data bilangan Heksadesimal (2 digit) berada pada register R20 menjadi desimal. Tempatkan hasil konversi pada Reg.R22:R21. Cara membuat program ini adalah dengan menggunakan perintah wajib terlebih dahulu, kemudian memasukan data yang diinginkan yaitu data bilangan heksadesimal 2 digit. Konversi bilangan heksadesimal ke desimal harus menggunakan cara dikalikan dengan 16. Setelah memasukan input nilai pada suatu register (r20), kemudian memindahkan kepada register yang lain (r25) agar register yang sebelumnya tidak rusak, setelah itu lakukan pengambilan nilai 16-an menggunakan LSR sebanyak 4 kali sehingga bergeser

kekanan. Setelah itu lakukan pemindahan register r25 ke r19, setelah itu lakukan penjulahan antara r19 dengan register yang belum digunakan sebelumnya (r30), kemudian memanggil subrutin daa untuk melakukan penjumlahan data secara desimal, setelah itu pindahkan hasil penjumlahan dan pengecekan kepada register yang dituju r21. Kemudian lakukan pengisian nilai suatu register yang belum terpakai dengan banyaknya perkalian yang akan dilakukan *looping*. Setelah itu buat sebuah subrutin penjumlahan berulang dengan cara melakukan pemindahan terlebih dahulu r21 ke r19 (ke r19 karena akan dilakukan pengecekan dan penjumlahan pada subrutin daa yang menggunakan r19), setelah itu jumlahkan r19, dengan r19, setelah dijumlahkan dicek terlebih dahulu kepada subrutin daa kemudian lakukan pemindahan register Kembali dari r19, ke r21. Setelah itu carry yang didapat saat proses penjumlahan antara r19,r19 disimpan menggunakan perintah adc, lakukan pengecekan Kembali di subrutin daa dan lakukan terus hingga penjumlahan berulangnya selesai. Setelah itu, dilanjut mengambil nilai satuan dari inputan datam kemudian lakukan penjumlahan nilainya dengan register yang nilainya masih 0, setelah itu cek di subrutin daa. Setelah itu, masukan carry dan jumlahkan, kemudian di cek di subrutin daa, kemudian pindahkan hasil kepada register yang diperintahkan r22. Setelah itu, lakukan perintah untuk mengakhiri suatu programnya. Lakukan *running* program dan liat hasilnya.

Pada program kedua yaitu Konversikan bilangan desimal (2 digit) berada pada register R16 menjadi hexadesimal. Tempatkan hasil konversi pada Reg.R17.

Cara membuat program ini adalah dengan menggunakan perintah wajib terlebih dahulu, kemudian memasukan data yang diinginkan yaitu data bilangan desimal 2 digit. Konversi bilangan desimal ke heksadesimal harus menggunakan cara dikalikan dengan 10 ((d1*0A)+d2). Setelah memasukan input nilai pada suatu register (r16), kemudian memindahkan kepada register (r20) yang lain agar register yang sebelumnya tidak rusak, setelah itu lakukan pengambilan nilai puluhan menggunakan LSR sebanyak 4 kali sehingga bergeser kekanan. Setelah itu masukan suatu register dengan nilai 10 (0A), setelah itu kalikan dengan register yang telah digeser (mul r20,r21), karena hasil perkalian pada ATmega 8535 itu disimpan di register r0, dan r1, maka harus dipindahkan terlebih dahulu register r0 ke register yang diminta. Setelah itu, lakukan pengambilan nilai satuan dari angka desimal itu, dan dijumlahkan dengan nilai di register r17. Hasilnya disimpan di r17 (output register yang diminta) Setelah itu, lakukan perintah untuk mengakhiri suatu programnya. Lakukan *running* program dan liat hasilnya.

Pada program ketiga yaitu Konversikan bilangan desimal (4 digit) berada pada

register R16 menjadi hexadecimal. Cara membuat program ini adalah dengan menggunakan perintah wajib terlebih dahulu, kemudian memasukan data yang diinginkan yaitu data bilangan desimal 4 digit. Konversi bilangan desimal ke heksadesimal harus menggunakan cara dikalikan dengan 10 ((((((d1*0A)+d2)*0A)+d3)*0A)+d4). Setelah memasukan input nilai pada data area, kemudian memindahkan kepada register (r18) yang lain agar register yang sebelumnya tidak rusak, setelah itu lakukan pengambilan nilai ribuan menggunakan LSR sebanyak 4 kali sehingga bergeser kekanan. Setelah itu masukan suatu register dengan nilai 10 (0A), setelah itu kalikan dengan register yang telah digeser, karena hasil perkalian pada ATmega 8535 itu disimpan di register r0, dan r1, maka harus dipindahkan terlebih dahulu register r0 ke register yang diminta. Setelah itu, lakukan pengambilan nilai ratusan, nilai yang sudah dikali 10 ditambahkan dengan nilai yang ratusan sebelumnya, beserta *carry*nya. Setelah itu, kalikan dengan register yang diminta, karena hasil perkalian pada ATmega 8535 itu disimpan di register r0, dan r1, maka harus dipindahkan terlebih dahulu register r0 ke register yang diminta. Selanjutnya, lakukan pengambilan nilai puluhan, kemudian digeser sebanyak 4 kali kekanan. Setelah itu, lakukan penjumlahan nilai yang disimpan ditempat yang diminta dengan nilai yang puluhan sebelumnya, beserta *carry*nya. Setelah itu, kalikan dengan register yang diminta, karena hasil perkalian pada ATmega 8535 itu disimpan di register r0, dan r1, maka harus dipindahkan terlebih dahulu register r0 ke register yang diminta. Kemudian, melakukan perkalian antara r22 dengan r31 (register yang belum terpakai). Lakukan pemanggilan subrutin yang telah dibuat untuk menambahkan nilai di r0 dan r1. Setelah itu, lakukan pengambilan nilai satuan, setelah itu lakukan penjumlahan dengan r21 beserta *carry*nya di r22. Hasilnya disimpan di r22, r21 (output register yang diminta) Setelah itu, lakukan perintah untuk mengakhiri suatu programnya. Lakukan *running* program dan liat hasilnya.

E. KESIMPULAN

Pada praktikum yang telah dilakukan, diketahui bahwa simulator yang digunakan yaitu ATmega 8535, diketahui pula cara kerjanya, serta memahami konsep dari bahasa pemograman Assembly. Pada praktikum ini pula, diketahui bahwa konversi suatu data heksadesimal ke desimal harus dikalikan dengan 16. Pada simulator ATmega 8535 tidak dapat menggunakan penjumlahan secara langsung, sehingga dibuat sebuah subrutin untuk menjumlahkan secara desimal. Kemudian, digunakan konsep pengambilan nilai suatu puluhannya saja, atau satuan, dan lainnya menggunakan perintah **ANDI** dan **LSR.** Kemudian, diketahui untuk mengonversikan suatu nilai desimal ke

heksadesimal harus dikalikan dengan 10, menggunakan konsep pengambilan nilai saatu tempat yang sama seperti dari heksadesimal ke desimal ((((((d1*0A)+d2)*0A)+d3)*0A)+d4). Kemudian, terdapat perintah **ADD** untuk menjumlahkan datanya dan **ADC** untuk menjumlahkan yang terdapat *carry*nya.

DAFTAR PUSTAKA

http://sistemkomputer.narotama.ac.id/2013/05/mikrokontroler-atmega8535/

http://christianto.tjahyadi.com/belajar-mikrokontroler/avr-studio-dan-winavr.html